

Dynamic Privacy-Preserving Proofs of Assets for Hidden On-Chain Address Sets

Anonymous Author(s)

Abstract. Proofs of solvency require a custodian to demonstrate that its assets cover its liabilities. The on-chain asset side is usually handled by publishing reserve addresses and proving control of the corresponding keys, while academic approaches such as Provisions study stronger privacy goals [6]. This is simple, but it reveals the reserve set and makes long-term treasury flows publicly traceable. A fully private alternative is to prove in zero knowledge that a hidden set of controlled addresses has enough balance under a public chain state root. The obstacle is dynamism: naively recomputing a large proof after every block is expensive, especially when the native chain state structure is not proof-friendly. We formalize dynamic private proof of assets for a hidden on-chain address set and give two concrete constructions. The first is an algebraic fixed-set update protocol. A scalar-masked Pedersen/IPA polynomial commitment binds the hidden reserve set through a randomized set polynomial, preserving zero tests while avoiding a structured trusted setup. For each public block-delta list, the prover commits to a hidden membership-status vector and to the hidden evaluation vector, and a single batched Bulletproof/IPA relation proves both that these evaluations come from the committed set polynomial and that the committed aggregate balance change is the inner product of the same hidden status vector with the public deltas. The second construction is a compact sparse-Merkle-tree implementation in a SNARK. It supports mutable reserve sets by proving membership, non-membership, conditional update, and insertion transitions directly over a circuit-friendly local tree. Both constructions bind every accepted asset commitment to a finalized chain state root, hide the reserve set and aggregate delta, and compose with a liability commitment through a threshold proof.

Keywords: Proof of assets · proof of solvency · zero knowledge · Bulletproofs · inner-product arguments · sparse Merkle trees

1 Introduction

A centralized exchange or custodian is solvent if it can cover its user liabilities. A cryptographic proof of solvency decomposes this claim into two parts. A proof of liabilities establishes the total amount owed to users, preferably without leaking individual user balances. A proof of assets establishes that the custodian controls enough assets. This paper focuses on the asset side for on-chain reserves.

The common on-chain method is to disclose reserve addresses. The custodian signs messages under the corresponding keys, and anyone can audit the asset total from the public chain state. This approach is easy to explain and easy to

verify, but it exposes the reserve set. Once the reserve set is public, observers can monitor inflows and outflows, identify counterparties, infer business activity, and analyze short-term liquidity management. These effects are not merely cosmetic: reserve-address disclosure can reveal operational strategy even when the custodian is honest and fully solvent.

A natural private alternative is to use a zero-knowledge proof. At a public state root sr_i , the prover would show that there exists a hidden set S of addresses, that the prover controls every address in S , that the balance of each address is correctly read from the chain state, and that a public commitment C_i opens to the aggregate balance

$$B_i = \sum_{a \in S} \text{bal}_{\text{sr}_i}(a).$$

This static statement is conceptually clean. It is also unsatisfactory as a long-term audit mechanism. First, if the native chain uses a proof-unfriendly state structure, then proving many native membership proofs inside a SNARK can dominate the cost. Second, a single proof at one time gives weak continuous accountability. A custodian may borrow assets shortly before the audit and return them afterwards. If the reserve set is hidden, users cannot track the reserve addresses directly.

We address this gap by making private proof of assets dynamic. The prover first produces an initialization proof under a finalized state root. Afterwards, every audit epoch is driven by a public delta list

$$\Delta_i = ((a_1, \delta_1), \dots, (a_m, \delta_m))$$

which records the balance changes between two finalized roots. The prover proves that the new aggregate commitment equals the old one plus exactly the deltas belonging to hidden reserve addresses:

$$B_i = B_{i-1} + \sum_{j=1}^m \mathbf{1}[a_j \in S] \delta_j.$$

The proof reveals neither S , nor $S \cap \{a_1, \dots, a_m\}$, nor the aggregate change $\sum_j \mathbf{1}[a_j \in S] \delta_j$. Every accepted commitment is tied to a concrete finalized root, so the verifier can follow a chain of private asset commitments over time.

Public synchronizer. The delta list is not a trusted oracle. It is the output of a deterministic public program `Sync` that reads finalized blocks, executes the chain’s public balance-transition rules, merges duplicate address entries, and outputs a canonical list of address-delta pairs. The verifier may run `Sync` locally, or verify a posted output by re-execution. The cryptographic proof does not hide or certify chain execution; it proves correct private-reserve evolution relative to this public, reproducible transformation of finalized blocks. This is the same trust boundary used by ordinary public reserve audits: correctness of the chain state is checked by the verifier’s chain policy, while the zero-knowledge layer protects the reserve set.

1.1 Contributions

We make the following contributions.

1. **Problem formulation.** We define dynamic private proof of assets for hidden on-chain reserve sets. The formulation captures initialization, incremental update, insertion of new reserve addresses, threshold comparison with liabilities, freshness with respect to state roots, and privacy of the hidden set and aggregate update.
2. **Algebraic fixed-set update.** We give a lightweight NIZK update protocol for a fixed hidden reserve set. The reserve set is committed by a scalar-masked Pedersen/IPA commitment to its set-polynomial coefficients. Each update proves a batched hidden multi-evaluation relation, a hidden zero-test relation on the public touched addresses, and a hidden inner-product relation against the public deltas. The prover uses fast multipoint evaluation and its transposed form, avoiding the naive $O(|S|m)$ evaluation cost and removing pairings and KZG quotient computations.
3. **Compact SMT-SNARK update.** We give a direct sparse-Merkle-tree implementation for mutable reserve sets. Instead of referring to a generic authenticated-data-structure interface, we specify the compact tree representation, membership and non-membership cases, update relation, insertion relation, and verifier logic.
4. **Security and costs.** We state the security games and prove soundness and privacy from the underlying Pedersen commitments, Bulletproof/IPA knowledge soundness, tree hashing, random-linear-combination evaluation binding, and synchronizer reproducibility. We also give asymptotic costs and an evaluation plan for a prototype.

2 Technical Overview

Why static private PoA is expensive. A static private proof can verify ownership of each hidden address and verify a native state proof for every hidden balance. This works for a one-time audit, but it repeats work that is independent of the small set of addresses touched in a later block. It also ties the proof circuit to the native chain state structure. Ethereum-style state proofs, Bitcoin UTXO proofs, and rollup state proofs have different formats and different circuit costs.

Dynamic update by public deltas. The key observation is that after initialization the reserve set changes slowly, while chain state evolves through public balance deltas. For a fixed set S , the only private question at update epoch i is which public touched addresses lie in S . If this hidden status vector is

$$u_j = \mathbf{1}[a_j \in S],$$

then the correct aggregate update is the single linear relation

$$d_i = \sum_{j=1}^m u_j \delta_j.$$

Therefore the update proof should bind u to the hidden set and bind the committed delta d_i to the same u , without revealing either.

Construction I. For a fixed reserve set, define the set polynomial

$$F_S(T) = \prod_{a \in S} (T - \text{Enc}(a))$$

and sample a secret nonzero scalar α . The committed polynomial is

$$P_S(T) = \alpha F_S(T) = \sum_{t=0}^n p_t T^t.$$

The zero set is unchanged: $a \in S$ iff $P_S(\text{Enc}(a)) = 0$, except with negligible encoding-collision probability. The public set accumulator is now a hiding Pedersen/IPA coefficient commitment

$$A = \sum_{t=0}^n p_t G_t + r_A H_A.$$

For the update list, the prover computes $y_j = P_S(x_j)$ for $x_j = \text{Enc}(a_j)$ and commits to the vector y with independent Pedersen bases. After the commitments are fixed, a Fiat–Shamir challenge β compresses all evaluation equations into

$$\sum_{t=0}^n p_t \left(\sum_{j=1}^m \beta^{j-1} x_j^t \right) = \sum_{j=1}^m \beta^{j-1} y_j.$$

A single committed-R1CS Bulletproof/IPA proves this batched evaluation equation, the zero-test constraints

$$u_j(1 - u_j) = 0, \quad u_j y_j = 0, \quad y_j z_j = w_j, \quad (1 - u_j)(w_j - 1) = 0,$$

and the balance-projection equation $d_i = \sum_j u_j \delta_j$. Thus the verifier learns that the public aggregate commitment was updated correctly but learns no membership bits or evaluation values, and the algebraic update uses no pairings and no KZG SRS.

Construction II. The algebraic construction is optimized for a fixed set. To support frequent insertion of new reserve addresses in a more conventional engineering stack, we also describe a compact sparse-Merkle-tree SNARK. The local tree contains only the hidden reserve addresses and their current balances. For every public touched address, the witness supplies either a membership path and an updated leaf, or a non-membership proof. The circuit recomputes the old root, applies the conditional transition, recomputes the new root, and checks the aggregate-commitment update. Insertion is a separate relation that proves ownership of the new address, chain-state consistency of its current balance, non-membership in the old tree, and a valid root transition.

3 Preliminaries

Let λ be the security parameter. All groups have prime order q , and all adversaries are probabilistic polynomial-time. We write $[m] = \{1, \dots, m\}$ and use additive notation for groups. We write $\tilde{O}(\cdot)$ to hide polylogarithmic factors.

Commitments. Let $V, H_B \in \mathbb{G}_1$ be independent generators. We commit to an aggregate balance by

$$\text{Com}_B(B; r) = BV + rH_B.$$

We use the hiding and binding properties of Pedersen commitments. Integer balances and deltas are encoded in $\mathbb{F}_q$, but the default protocol proves range constraints for balances, deltas, and updated balances so that accepted field equations have the intended integer semantics. A public modulus-safety bound can be used only as an implementation optimization.

Pedersen/IPA polynomial commitments. Let $\mathbb{G}$ be a prime-order group with public generators

$$G_0, \dots, G_D, H_A \in \mathbb{G}$$

derived by domain-separated hash-to-curve, so that no efficient adversary knows a nontrivial linear relation among them. A degree-at-most- D polynomial

$$f(T) = \sum_{t=0}^D f_t T^t$$

is committed by the hiding coefficient commitment

$$\text{PC}(f; r) = \sum_{t=0}^D f_t G_t + rH_A.$$

This is a Pedersen vector commitment to the coefficient vector. We use its computational binding under the discrete-log/no-known-relation assumption and its perfect hiding from the blinding term.

For query points $x_1, \dots, x_m$, the prover may commit to a hidden evaluation vector by

$$C_Y = \sum_{j=1}^m y_j Y_j + r_Y H_Y,$$

where $(Y_j)_j, H_Y$ are independent generators derived from the public update transcript. After the coefficient commitment A , the value commitment C_Y , and all other update commitments are fixed, the verifier derives a Fiat–Shamir challenge $\beta \in \mathbb{F}_q$ and sets

$$\gamma_j = \beta^{j-1}, \quad b_t^* = \sum_{j=1}^m \gamma_j x_j^t.$$

The m evaluation equations $y_j = f(x_j)$ are then compressed into the single linear identity

$$\sum_{t=0}^D f_t b_t^* = \sum_{j=1}^m \gamma_j y_j.$$

If at least one claimed value is wrong, the difference is a nonzero polynomial in β of degree at most $m - 1$, so the batched identity passes with probability at most $(m - 1)/q$ in the interactive protocol, and in the random-oracle model after Fiat–Shamir. The identity and the openings of A and C_Y are proved by a committed-R1CS Bulletproof whose final check is an inner-product argument. We call this the Bulletproof/IPA multi-evaluation proof.

Scalar-masked set commitments. The algebraic construction commits to

$$P_S(T) = \alpha \prod_{a \in S} (T - \text{Enc}_{\mathbb{F}}(a))$$

for a fresh secret $\alpha \in \mathbb{F}_q^*$, rather than to the monic set polynomial itself. Multiplication by $\alpha \neq 0$ does not change the roots, so it preserves the membership predicate used by the update proof. In the Bulletproof/IPA version, the public accumulator is the hiding coefficient commitment $A = \text{PC}(P_S; r_A)$. Its privacy comes from the Pedersen blinding $r_A H_A$; the scalar mask is kept because it makes the zero-test polynomial non-monic and ensures that the initialization relation does not expose a canonical set polynomial even inside split implementations.

Committed-R1CS Bulletproofs. Bulletproofs [2] give short inner-product proofs; in our use, a committed-R1CS Bulletproof proves knowledge of variables that satisfy arithmetic constraints and open public multi-generator commitments. We need the frontend to support equations of the form

$$C = \sum_j a_j G_j + rH,$$

where $(G_j)_j, H$ are public group elements, and public linear equations such as $\langle a, b \rangle = \langle y, \gamma \rangle$. The proof binds the same witness variables to the external commitments and to the R1CS constraints. The final proof is logarithmic-size in the number of multiplication gates and committed-vector length, with linear prover work and linear verifier multi-scalar multiplication work.

Sparse Merkle trees. Sparse Merkle trees are a standard hash-based authenticated structure [4]. The SMT construction uses a depth- ℓ binary tree over keys $\kappa \in \{0, 1\}^\ell$. Empty subtree hashes are fixed public values $E_0, \dots, E_\ell$. A leaf containing key κ , balance b , and a long random leaf salt η is

$$L = \text{H}(\text{leaf}, \kappa, b, \eta),$$

where H is a circuit-friendly collision-resistant compression function. An internal node at level d is

$$N = \text{H}(\text{node}, d, N_0, N_1).$$

The prover stores the tree compactly: default subtrees are not materialized, and only non-default branches are kept. Membership and non-membership witnesses are private circuit witnesses. The salt prevents the public root from becoming a deterministic commitment to small candidate address sets.

Public synchronizer. For finalized roots $\text{sr}_{i-1}, \text{sr}_i$, the deterministic public program

$$\Delta_i \leftarrow \text{Sync}(\text{sr}_{i-1}, \text{sr}_i)$$

outputs a canonical list of balance changes $((a_1, \delta_1), \dots, (a_m, \delta_m))$. Canonical means that addresses are uniquely encoded, sorted, and deduplicated. Duplicate effects on the same address are merged into one integer delta. The verifier accepts an update proof only for a delta list equal to the output of this public program under its finality policy.

4 Model and Security Definitions

4.1 System state

We first define the single-chain problem. At epoch i , the public state is

$$X_i = (\text{sr}_i, R_i, C_i),$$

where sr_i is a finalized chain state root, R_i is either the fixed Pedersen/IPA set-polynomial commitment A or the SMT root, and $C_i = \text{Com}_B(B_i; r_i)$ is a commitment to the aggregate reserve balance. The private state contains the reserve set S , the opening (B_i, r_i) , and either the masked set polynomial P_S and its unmasked zero set, or the compact salted SMT state. The intended aggregate is

$$B_i = \sum_{a \in S} \text{bal}_{\text{sr}_i}(a).$$

A liability proof is external to this paper. We assume the verifier has either a public liability value L_i or a liability commitment together with a zero-knowledge proof that it opens to L_i . The asset proof composes with it by proving $B_i \geq L_i$ or, more generally, $B_i - L_i \geq 0$ for committed values.

4.2 Algorithms

A dynamic private PoA scheme consists of the following algorithms.

- **Setup**(1^λ) outputs public parameters.
- **Init** takes a finalized root sr_0 and private reserve set data. It outputs X_0 and a proof π_{init} .
- **Update** takes an accepted state X_{i-1} , a new finalized root sr_i , the public delta list $\Delta_i = \text{Sync}(\text{sr}_{i-1}, \text{sr}_i)$, and private state. It outputs X_i and π_{upd} .

- **Insert** adds a new reserve address. It proves address ownership, current chain-state consistency of the inserted balance, non-membership before insertion, and a valid root and commitment transition.
- **Verify** checks finality, recomputes or verifies the public synchronizer output, verifies the zero-knowledge proof, and checks that the old state referenced by an update is the last accepted state.
- **ProveThr** proves a threshold statement over the current balance commitment.

4.3 Security properties

Definition 1 (Initialization soundness). *A scheme has initialization soundness if no efficient adversary can produce an accepted (X_0, π_{init}) such that the hidden state contains an address not controlled by the prover, a balance not equal to the balance under sr_0 , a malformed reserve-set commitment/root, or a balance commitment not equal to the aggregate induced by the hidden reserve set.*

Definition 2 (Update soundness). *A scheme has update soundness if no efficient adversary can produce an accepted transition from X_{i-1} to X_i for a public delta list Δ_i such that*

$$C_i \neq C_{i-1} + \text{Com}_B \left(\sum_{(a, \delta) \in \Delta_i : a \in S} \delta; \rho_i \right)$$

for any valid blinding difference ρ_i , where S is the reserve set bound by the previous accepted state. In the SMT construction, update soundness also requires that the new root is the correct root obtained by applying exactly the conditional updates for touched reserve addresses.

Definition 3 (Threshold soundness). *A scheme has threshold soundness if no efficient adversary can convince the verifier to accept a threshold proof for (C_i, T) when every valid opening (B_i, r_i) of C_i satisfies $B_i < T$.*

Definition 4 (Privacy). *Address privacy and balance privacy are simulation-based. The verifier's view consists of finalized roots, public delta lists, public roots/commitments, threshold statements, and proof strings. For any two executions with the same public leakage and the same truth value of the threshold predicate, the views are computationally indistinguishable. In particular, the verifier learns neither the reserve set, nor the touched reserve subset, nor the exact aggregate balance or aggregate delta beyond what follows from accepted threshold statements.*

Definition 5 (Freshness). *A scheme has freshness if an accepted asset statement is bound to the state root named in the public state. A proof for an older root cannot be reused for a newer root unless there is an accepted chain of update proofs linking the old public state to the new one.*

5 Construction I: Algebraic Fixed-Set Update via Bulletproof/IPA

This construction is optimized for a fixed reserve set. It is the pairing-free version of the algebraic update protocol: the KZG accumulator and KZG multi-opening are replaced by a Pedersen coefficient commitment and a batched Bulletproof/IPA multi-evaluation proof. It is suitable when the custodian initializes a reserve set and then refreshes the aggregate balance frequently. Address-set extension is possible but is not the fast path; the mutable-set construction in Section 6 is better suited for frequent insertions.

5.1 State and initialization

Let S be the hidden address set and let $x_a = \text{Enc}_{\mathbb{F}}(a)$. Define

$$F_S(T) = \prod_{a \in S} (T - x_a), \quad P_S(T) = \alpha F_S(T) = \sum_{t=0}^n p_t T^t$$

for a secret $\alpha \leftarrow \mathbb{F}_q^*$. The public set accumulator is the Pedersen/IPA coefficient commitment

$$A = \sum_{t=0}^n p_t G_t + r_A H_A.$$

The roots of P_S are exactly the encoded reserve addresses, while the blinding r_A and the scalar α hide the polynomial coefficients. At epoch i , the public state is

$$X_i = (\text{sr}_i, A, C_i), \quad C_i = \text{Com}_B(B_i; r_i).$$

The private state is $(S, \alpha, F_S, P_S, (p_t)_{t=0}^n, r_A, B_i, r_i)$.

The initialization proof is the only place where the prover proves that the hidden reserve set is a real controlled on-chain set under sr_0 . Afterwards the verifier keeps only the public accumulator A and the aggregate commitment C_0 . We write the relation below for a fixed public size bound n . If n is not fixed by the system parameters, it is included in the public initialization statement and in the Fiat–Shamir transcript; it is then part of the intentional leakage.

Initialization statement. Let the hidden reserve list be $(a_1, \dots, a_n)$, let

$$x_k = \text{Enc}_{\mathbb{F}}(a_k), \quad b_k = \text{bal}_{\text{sr}_0}(a_k),$$

and let $p_0, \dots, p_n$ be the coefficients of P_S . The public input is (sr_0, A, C_0, n) , with n omitted when fixed globally. The witness contains

$$((a_k, x_k, b_k, \omega_k^{\text{own}}, \pi_k^{\text{chain}})_{k=1}^n, \alpha, (p_t)_{t=0}^n, r_A, B_0, r_0).$$

The relation requires the following facts.

1. **Ownership.** For every k , the private proof ω_k^{own} satisfies an address-ownership predicate

$$\text{Own}(a_k, \omega_k^{\text{own}}; \text{ch}_{\text{init}}) = 1,$$

where $\text{ch}_{\text{init}} = \text{H}(\text{init}, \text{sr}_0, A, C_0, n)$. For an externally owned account this can be instantiated by verifying a signature on ch_{init} and checking that the verification key derives to a_k ; contract-wallet predicates can be treated as a chain-specific ownership relation.

2. **Native state consistency.** For every k , the native chain proof verifies

$$\text{ChainVerify}(\text{sr}_0, a_k, b_k, \pi_k^{\text{chain}}) = 1.$$

Thus b_k is exactly the balance of the hidden address under the public finalized root sr_0 . The proof also range-checks b_k so that later field equations represent integer balances.

3. **No duplicate reserve address.** The relation checks that the hidden addresses are distinct. A direct implementation sorts the address bitstrings and proves $a_1 < \dots < a_n$. A hash-tree implementation instead inserts the hidden keys into a private SMT and proves that every insertion was into an empty position. This check is necessary: otherwise the prover could count the same controlled address more than once in B_0 .
4. **Algebraic set binding.** The hidden polynomial bound by A must be the scalar-masked set polynomial

$$P(T) = \alpha \prod_{k=1}^n (T - x_k), \quad \alpha \neq 0,$$

and the accumulator must be the Pedersen coefficient commitment

$$A = \sum_{t=0}^n p_t G_t + r_A H_A.$$

This is a group-linear opening relation in the private coefficients and blinding.

5. **Aggregate balance binding.** The aggregate and its commitment satisfy

$$B_0 = \sum_{k=1}^n b_k, \quad C_0 = B_0 V + r_0 H_B.$$

Efficient product check. A literal circuit for the coefficient identity $P(T) = \alpha \prod_k (T - x_k)$ can be implemented by a product tree, but it is unnecessary for initialization soundness. The prover first fixes the public commitments (A, C_0) and the hidden-vector bindings used by the proof system, and the verifier derives

$$\zeta = \text{H}(\text{init-poly}, \text{sr}_0, A, C_0, n) \in \mathbb{F}_q.$$

The proof checks only the random evaluation identity

$$\sum_{t=0}^n p_t \zeta^t = \alpha \prod_{k=1}^n (\zeta - x_k).$$

Inside the circuit this costs $O(n)$ multiplication gates by the recurrence

$$\begin{aligned} g_0 &= 1, \\ g_k &= g_{k-1}(\zeta - x_k), \\ \hat{P} &= \sum_{t=0}^n p_t \zeta^t, \end{aligned}$$

followed by $\hat{P} = \alpha g_n$. Since the Pedersen commitment A binds the coefficient vector before ζ is derived, a false degree-at-most- n product identity passes with probability at most n/q . This turns the expensive coefficient-product check into a linear-size randomized algebraic check. If one wants deterministic checking, the same relation can replace this step by a product-tree coefficient computation at $\tilde{O}(n)$ arithmetic cost outside the circuit and $O(n \log n)$ or larger in-circuit cost, depending on the circuit model.

Split implementation. The initialization proof may be realized as one monolithic SNARK/ZKVM proof, but a more efficient implementation is a two-part proof

$$\pi_{\text{init}} = (C_X, C_B, C_P, \pi_{\text{chain}}, \pi_{\text{alg}}).$$

Here

$$C_X = \sum_{k=1}^n x_k G_k^X + \rho_X H_X, \quad C_B = \sum_{k=1}^n b_k G_k^B + \rho_B H'_B,$$

and

$$C_P = A = \sum_{t=0}^n p_t G_t + r_A H_A$$

are hiding vector commitments to encoded addresses, balances, and set-polynomial coefficients. The ZKVM proof π_{chain} verifies the native chain proofs, ownership predicates, duplicate-freeness, and the openings of C_X, C_B . The algebraic proof π_{alg} opens the same C_X, C_B , opens A to $(p_t)_t$, proves the randomized product check above, and proves C_0 commits to $\sum_k b_k$. The public commitments are the glue that prevents the two subproofs from using different hidden address, balance, or polynomial data. They are hiding, so they do not reveal the reserve set.

Algorithm 1 Bulletproof/IPA algebraic initialization protocol

Require: Public finalized root sr_0 ; private reserve witnesses $(a_k, \omega_k^{\text{own}}, \pi_k^{\text{chain}})_{k=1}^n$.

- 1: For each k , compute $x_k = \text{Enc}_{\mathbb{F}}(a_k)$ and read b_k from the native chain proof.
 - 2: Sample $\alpha \leftarrow \mathbb{F}_q^*$ and compute $P(T) = \alpha \prod_k (T - x_k) = \sum_{t=0}^n p_t T^t$ using a product tree.
 - 3: Sample $r_A, r_0 \leftarrow \mathbb{F}_q$, set $A = \sum_t p_t G_t + r_A H_A$, $B_0 = \sum_k b_k$, and $C_0 = B_0 V + r_0 H_B$.
 - 4: Generate π_{init} proving ownership, native state consistency, duplicate-freeness, the randomized product check, the opening of A , and the opening of C_0 .
 - 5: Output $X_0 = (\text{sr}_0, A, C_0)$, proof π_{init} , and private state $(S, \alpha, F_S, P_S, (p_t)_t, r_A, B_0, r_0)$.
-

Initialization cost. Let c_{chain} be the cost of one hidden native state proof plus one hidden ownership proof. The monolithic relation costs $O(nc_{\text{chain}})$ plus $O(n)$ arithmetic constraints for summation and the randomized product check, and one group-linear opening proof for the Pedersen coefficient commitment. The prover may compute the coefficients of P outside the circuit using a product tree in $\tilde{O}(n)$ polynomial work. In the split implementation, the ZKVM handles the chain-specific work, while the algebraic proof handles only vector openings, the product evaluation check, and aggregate commitment. This is usually preferable because native state verification uses chain-specific primitives such as Merkle-Patricia hashing and signature verification, whereas the algebraic part is small and curve-friendly.

5.2 Update proof

Let the public update list be

$$\Delta_i = ((a_1, \delta_1), \dots, (a_m, \delta_m)),$$

where the a_j 's are distinct. Set $x_j = \text{Enc}_{\mathbb{F}}(a_j)$. The prover computes

$$y_j = P_S(x_j), \quad u_j = \begin{cases} 1 & y_j = 0, \\ 0 & y_j \neq 0, \end{cases}$$

and

$$d_i = \sum_{j=1}^m u_j \delta_j.$$

The proof must show that y is the evaluation vector of the committed polynomial, that u is exactly the membership vector, and that d_i is computed from the same u .

Let $(U_1, \dots, U_m, Y_1, \dots, Y_m, H_U, H_Y, V, H_B) \in \mathbb{G}$ be domain-separated public generators. The bases $(Y_j)_j, H_Y$ are derived from the update transcript, e.g.

$$Y_j = \text{HashToCurve}(\text{eval-base}, \text{sr}_{i-1}, \text{sr}_i, A, C_{i-1}, j),$$

so that the verifier does not store per-update bases. The prover samples $r_U, r_Y, r_D \leftarrow \mathbb{F}_q$ and computes

$$C_U = \sum_{j=1}^m u_j U_j + r_U H_U, \quad C_Y = \sum_{j=1}^m y_j Y_j + r_Y H_Y, \quad C_D = d_i V + r_D H_B.$$

The new balance commitment is defined by homomorphism:

$$C_i = C_{i-1} + C_D.$$

Batched multi-evaluation equation. After (A, C_U, C_Y, C_D, C_i) and the canonical update list are fixed, the verifier derives

$$\beta = H(\mathbf{bp}\text{-eval}, X_{i-1}, X_i, \Delta_i, C_U, C_Y, C_D) \in \mathbb{F}_q.$$

Let

$$\gamma_j = \beta^{j-1}, \quad b_t^* = \sum_{j=1}^m \gamma_j x_j^t \quad (0 \leq t \leq n).$$

Then the multi-evaluation relation is compressed to

$$\sum_{t=0}^n p_t b_t^* = \sum_{j=1}^m \gamma_j y_j.$$

The verifier computes γ and b^* from the public list. The prover computes y by fast multipoint evaluation and can compute b^* either by the transposed multipoint-evaluation algorithm or, for structured domains, by FFT/NTT routines.

Merged logic and evaluation proof. The prover gives one committed-R1CS Bulletproof/IPA proof $\pi_{\text{upd}}^{\text{bp}}$ for witness

$$((p_t)_{t=0}^n, u, y, z, w, d_i, r_A, r_U, r_Y, r_D)$$

and public commitments (A, C_U, C_Y, C_D) . The proof includes the group-linear opening equations

$$A = \sum_{t=0}^n p_t G_t + r_A H_A,$$

$$C_U = \sum_j u_j U_j + r_U H_U, \quad C_Y = \sum_j y_j Y_j + r_Y H_Y, \quad C_D = d_i V + r_D H_D,$$

the batched evaluation equation

$$\sum_{t=0}^n p_t b_t^* - \sum_{j=1}^m \gamma_j y_j = 0,$$

the zero-test constraints

$$u_j(1 - u_j) = 0, \quad u_j y_j = 0, \quad y_j z_j = w_j, \quad (1 - u_j)(w_j - 1) = 0,$$

and the balance-projection constraint

$$d_i - \sum_{j=1}^m u_j \delta_j = 0.$$

Range constraints prove integer safety for d_i and for the updated aggregate value $B_{i-1} + d_i$.

Algorithm 2 Bulletproof/IPA algebraic fixed-set update**Require:** Old state $X_{i-1} = (\mathbf{sr}_{i-1}, A, C_{i-1})$, new root $\mathbf{sr}_i$, and $\Delta_i = \text{Sync}(\mathbf{sr}_{i-1}, \mathbf{sr}_i)$.**Require:** Witness $(S, \alpha, F_S, P_S, (p_t)_t, r_A, B_{i-1}, r_{i-1})$.

- 1: For each $(a_j, \delta_j) \in \Delta_i$, set $x_j = \text{Enc}_{\mathbb{F}}(a_j)$, compute $y_j = P_S(x_j)$, and set $u_j = 1[y_j = 0]$.
- 2: Compute $d_i = \sum_j u_j \delta_j$.
- 3: Compute $C_U = \sum_j u_j U_j + r_U H_U$, $C_Y = \sum_j y_j Y_j + r_Y H_Y$, $C_D = d_i V + r_D H_B$, and $C_i = C_{i-1} + C_D$.
- 4: Derive β , compute $\gamma_j = \beta^{j-1}$ and $b_t^* = \sum_j \gamma_j x_j^t$.
- 5: Generate one committed-R1CS Bulletproof $\pi_{\text{upd}}^{\text{bp}}$ for the group openings, the batched evaluation equation, the zero-test constraints, and the balance-projection constraint.
- 6: Output $X_i = (\mathbf{sr}_i, A, C_i)$ and $\pi_i^{\text{alg}} = (C_U, C_Y, C_D, \pi_{\text{upd}}^{\text{bp}})$.

Algorithm 3 Verifier for Bulletproof/IPA algebraic update**Require:** Old state X_{i-1} , claimed new state X_i , proof π_i^{alg} , and roots $\mathbf{sr}_{i-1}, \mathbf{sr}_i$.

- 1: Check that $\mathbf{sr}_i$ is finalized and that $\Delta_i = \text{Sync}(\mathbf{sr}_{i-1}, \mathbf{sr}_i)$ is canonical.
- 2: Check that the set-polynomial commitment A is unchanged and that $C_i = C_{i-1} + C_D$.
- 3: Compute $x_j = \text{Enc}_{\mathbb{F}}(a_j)$, check that the x_j 's are distinct, derive $(Y_j)_j, H_Y$, derive β , and compute γ and b^* .
- 4: Verify $\pi_{\text{upd}}^{\text{bp}}$ against the public statement containing $A, C_U, C_Y, C_D, \Delta_i, (G_t)_t, H_A, (U_j)_j, H_U, (Y_j)_j, H_Y, V, H_B, b^*, \gamma$.
- 5: Accept iff all checks pass.

Optimizations. The update proof is optimized in four ways. First, the KZG interpolation polynomial, quotient polynomial, and pairing check are removed entirely; the only polynomial work is multipoint evaluation and transposed multipoint evaluation. Second, the evaluation binding equation and the membership/delta logic are merged into one Bulletproof so that the commitments C_Y and C_U are opened only once. Third, the verifier derives all update-specific bases by hash-to-curve from the transcript; only the universal coefficient bases $(G_t)_t$ are fixed in the public parameters. Fourth, when the touched addresses lie in a structured evaluation domain, the prover computes $(y_j)_j$ and $(b_t^*)_t$ with FFT/NTT routines; for arbitrary points it uses subproduct trees and the transposed remainder tree.

Cost. Let $n = |S|$ and $m = |\Delta_i|$. The prover stores P_S in coefficient, FFT/NTT, or product-tree form. The evaluation vector $(P_S(x_1), \dots, P_S(x_m))$ is computed by fast multipoint evaluation. The batching vector $(b_t^*)_{t=0}^n$ is computed by transposed multipoint evaluation. For arbitrary points this is $\tilde{O}(n + m)$ in the usual polynomial-arithmetic model, with a conservative implementation bound such as $O((n + m) \log^2(n + m))$; for structured FFT domains it is close to $O(n \log n + m \log m)$. The Bulletproof relation has $4m$ multiplication gates for zero tests, linear constraints for evaluation and balance projection, and range constraints

for integer safety. The proof contains no pairing elements and has $O(\log(n + m + g_{\text{range}}))$ group elements, where g_{range} is the number of range-proof gates. Prover group work is linear in the committed-vector length and gate count, and verifier work is dominated by linear-size multi-scalar multiplications for the Bulletproof check.

6 Construction II: Compact SMT-SNARK Update

The second construction is designed for deployment and mutable reserve sets. It uses a circuit-friendly local sparse Merkle tree over the hidden reserve addresses. The tree is not the native chain state tree. Native chain proofs are used only during initialization and insertion. Incremental updates use the public synchronizer output and the local tree.

6.1 Tree state

For an address a , let $\kappa = \text{Enc}_{\text{key}}(a) \in \{0, 1\}^\ell$. A non-empty reserve leaf stores (κ, b, η) , where b is the current balance of the reserve address and η is a random leaf salt. The leaf hash is

$$L = H(\text{leaf}, \kappa, b, \eta).$$

The public state is

$$X_i = (\text{sr}_i, R_i, C_i),$$

where R_i is the SMT root and $C_i = \text{Com}_B(B_i; r_i)$. The private state is the compact tree, the reserve set, the leaf balances, the leaf salts, and the opening (B_i, r_i) .

6.2 Initialization

The SMT initialization proof has public input $(\text{sr}_0, R_0, C_0, n)$, with n omitted if fixed by the system parameters. The witness contains the hidden reserve addresses, ownership witnesses, native chain-state proofs, balances, leaf salts, and a compact representation of the local SMT. It proves that the public SMT root is a salted commitment to exactly the controlled addresses whose native balances are read from sr_0 .

More explicitly, for each hidden address a_k , the circuit computes

$$\kappa_k = \text{Enc}_{\text{key}}(a_k), \quad L_k = H(\text{leaf}, \kappa_k, b_k, \eta_k),$$

where b_k is verified by the native chain proof and η_k is a fresh private salt. The compact tree is then reconstructed from the sorted keys $(\kappa_k)_k$ and leaves $(L_k)_k$. The circuit checks that the keys are strictly ordered, or equivalently that each private insertion is performed at an empty position. This simultaneously

proves duplicate-freeness and binds every non-empty leaf of the local SMT to one hidden reserve address. Finally it checks

$$R_0 = \text{root}(\{(\kappa_k, b_k, \eta_k)\}_{k=1}^n), \quad B_0 = \sum_{k=1}^n b_k, \quad C_0 = B_0 V + r_0 H_B.$$

Algorithm 4 SMT initialization relation $\mathcal{R}_{\text{init}}^{\text{smt}}$

Require: Public input $(\text{sr}_0, R_0, C_0, n)$; witness $(a_k, b_k, \eta_k, \omega_k^{\text{own}}, \pi_k^{\text{chain}})_{k=1}^n$, compact tree data, and (B_0, r_0) .

- 1: **for** $k = 1$ to n **do**
 - 2: Verify $\text{Own}(a_k, \omega_k^{\text{own}}; \text{ch}_{\text{init}}) = 1$.
 - 3: Verify $\text{ChainVerify}(\text{sr}_0, a_k, b_k, \pi_k^{\text{chain}}) = 1$.
 - 4: Compute $\kappa_k = \text{Enc}_{\text{key}}(a_k)$ and $L_k = \text{H}(\text{leaf}, \kappa_k, b_k, \eta_k)$.
 - 5: Range-check b_k .
 - 6: Check that the keys $(\kappa_k)_k$ are duplicate-free, by sorted order or by private empty-position insertions.
 - 7: Recompute the compact SMT root from $(\kappa_k, L_k)_k$ and the public default hashes $(E_d)_d$; check that it equals R_0 .
 - 8: Check $B_0 = \sum_{k=1}^n b_k$, $C_0 = B_0 V + r_0 H_B$, and the range bound on B_0 .
-

This step is intentionally allowed to be heavy. Its purpose is to bind the hidden reserve set and local tree to a real chain state root. Later updates do not repeat native chain membership proofs. In an efficient implementation, native chain verification and ownership checking can again be placed in a ZKVM subproof, while a smaller SNARK checks the circuit-friendly SMT construction and aggregate commitment. The two subproofs must share hiding commitments to the same key and balance vectors, just as in the split algebraic initialization; otherwise the chain proof and the local-tree proof could refer to different hidden addresses.

6.3 Update relation

For a public delta list $\Delta_i = ((a_1, \delta_1), \dots, (a_m, \delta_m))$, the witness contains, for every j , one of two cases.

Membership case. The address is in the reserve tree. The witness contains the old balance b_j , the leaf salt η_j , the old membership path, and the new balance

$$b'_j = b_j + \delta_j.$$

The circuit recomputes the old path using the salted leaf $\text{H}(\text{leaf}, \kappa_j, b_j, \eta_j)$ and the new path after replacing it by $\text{H}(\text{leaf}, \kappa_j, b'_j, \eta_j)$. The bit u_j is set to 1.

Algorithm 5 SMT update relation $\mathcal{R}_{\text{upd}}^{\text{smt}}$

Require: Public input (X_{i-1}, X_i, Δ_i) , where $X_{i-1} = (\text{sr}_{i-1}, R_{i-1}, C_{i-1})$, $X_i = (\text{sr}_i, R_i, C_i)$.

Require: Witness: compact old tree, membership/non-membership witnesses for each public touched address, old balances and salts for member leaves, collision-leaf data for non-membership witnesses, and blinding difference ρ_i .

- 1: Set running root accumulator $\hat{R} \leftarrow R_{i-1}$ and $d \leftarrow 0$.
- 2: **for** $j = 1$ to m **do**
- 3: Set $\kappa_j = \text{Enc}_{\text{key}}(a_j)$.
- 4: Verify either a membership witness or a non-membership witness for κ_j against the current compact root representation.
- 5: **if** membership case **then**
- 6: Check old salted leaf (κ_j, b_j, η_j) , compute $b'_j = b_j + \delta_j$, and range-check b'_j .
- 7: Update the compact path and set $\hat{R}$ to the resulting root.
- 8: Set $d \leftarrow d + \delta_j$.
- 9: **else**
- 10: Leave $\hat{R}$ unchanged.
- 11: Check $\hat{R} = R_i$.
- 12: Check $C_i - C_{i-1} = dV + \rho_i H_B$.
- 13: Check integer range constraints for d and updated balances.

Non-membership case. The address is not in the reserve tree. The witness contains either a default-subtree witness, showing that the search path reaches a default subtree, or a collision-leaf witness, showing that the search path reaches a different salted leaf with key $\kappa' \neq \kappa_j$. The circuit checks the non-membership witness against the old root. The root is unchanged for this address and $u_j = 0$.

The circuit processes the touched addresses in canonical order. If several membership updates affect different leaves, it uses a compact multiproof representation to share common path nodes. The circuit checks that no address is omitted or duplicated because it receives exactly the public canonical list Δ_i .

The balance delta is

$$d_i = \sum_{j=1}^m u_j \delta_j.$$

The commitment check is

$$C_i = C_{i-1} + d_i V + \rho_i H_B,$$

where $\rho_i = r_i - r_{i-1}$ is private. The final root after all conditional updates must equal the public R_i .

6.4 Insertion relation

To insert a new reserve address a_{new} , the prover supplies an ownership proof, a chain proof showing current balance b under the current root sr_i , and an SMT

Algorithm 6 SMT insertion relation $\mathcal{R}_{\text{ins}}^{\text{smt}}$

Require: Public input $(\text{sr}_i, R_i, C_i, R'_i, C'_i)$; witness $(a, b, \eta, \omega^{\text{own}}, \pi^{\text{chain}}, \pi^{\text{non}}, \rho)$.

- 1: Verify ownership proof ω^{own} for a .
 - 2: Verify chain proof π^{chain} that $b = \text{bal}_{\text{sr}_i}(a)$.
 - 3: Verify non-membership of $\kappa = \text{Enc}_{\text{key}}(a)$ in the old tree root R_i .
 - 4: Sample or witness a fresh salt η , insert leaf $H(\text{leaf}, \kappa, b, \eta)$, and recompute root R'_i .
 - 5: Check $C'_i - C_i = bV + \rho H_B$ and range-check b .
-

non-membership proof for $\kappa = \text{Enc}_{\text{key}}(a_{\text{new}})$ under R_i . The circuit inserts the new leaf, computes the new root R'_i , and checks

$$C'_i = C_i + bV + \rho H_B.$$

Deletion can be supported similarly, but we keep deletion out of the main construction because reserve proofs usually benefit from monotone expansion: if an exchange wants to stop tracking an address, it can move its balance to another tracked address and then use an update epoch to reduce the old leaf to zero.

6.5 Verifier logic

The verifier accepts an SMT update only if sr_i is finalized, $\Delta_i = \text{Sync}(\text{sr}_{i-1}, \text{sr}_i)$ is canonical, the old public state equals the last accepted state, and the SNARK proof verifies for $\mathcal{R}_{\text{upd}}^{\text{smt}}$. Insertion and threshold proofs are checked against the current accepted state. The verifier never receives raw reserve addresses, per-address tree paths, or balances.

Cost. Let $m = |\Delta_i|$, $t \leq m$ be the number of touched reserve addresses, and ℓ be the tree depth. Independent paths cost $O(m\ell)$ hash checks. A compact multiproof shares common path nodes and costs $O(N_{\text{frontier}})$ hash checks, where $N_{\text{frontier}} \leq O(m\ell)$ is the number of distinct non-default nodes touched by the update. The SNARK verifier is succinct. The prover cost is dominated by the circuit-friendly hash constraints for the compact multiproof and by range checks for updated balances.

7 Security Analysis

We prove the main claims for accepted public states. The statements are relative to the public synchronizer output: the verifier recomputes $\Delta_i = \text{Sync}(\text{sr}_{i-1}, \text{sr}_i)$ and accepts only canonical, deduplicated lists. For the algebraic construction, Fiat–Shamir is modeled as a random oracle, or equivalently the protocol can be read first as an interactive public-coin protocol.

Theorem 1 (Initialization soundness). *Assume soundness of the initialization SNARK or ZKVM, soundness of the address-ownership and native chain-state proofs, binding of the Pedersen coefficient commitment for the algebraic*

construction, collision resistance of $\mathbf{H}$ for the SMT construction, binding of the balance commitment, and integer range soundness. If an initialization proof for X_0 accepts, then there exists a reserve set S controlled by the prover such that the public root or accumulator binds to S , and C_0 opens to

$$\sum_{a \in S} \text{bal}_{\text{sr}_0}(a).$$

Proof. SNARK/Bulletproof knowledge soundness extracts a witness satisfying the initialization relation. Ownership-proof soundness gives control of every extracted address. Chain-proof soundness gives $b_0(a) = \text{bal}_{\text{sr}_0}(a)$. The duplicate-freeness check ensures that the extracted list represents a set rather than a multiset. In the algebraic construction, the relation checks $\alpha \neq 0$, the group-linear opening equation

$$A = \sum_t p_t G_t + r_A H_A,$$

and the randomized identity

$$\sum_t p_t \zeta^t = \alpha \prod_{a \in S} (\zeta - \text{Enc}_{\mathbb{F}}(a)).$$

Pedersen binding fixes the coefficient vector represented by $(p_t)_t$ before ζ is derived. If this polynomial were not $\alpha \prod_{a \in S} (T - \text{Enc}_{\mathbb{F}}(a))$, the difference would be a nonzero polynomial of degree at most n , and the random evaluation check would pass with probability at most n/q . In the SMT construction, the relation recomputes the salted leaves and the root; collision resistance prevents inconsistent leaf assignments for the same root. In both constructions, the relation checks $C_0 = B_0 V + r_0 H_B$ with $B_0 = \sum_{a \in S} b_0(a)$, and Pedersen binding fixes the committed aggregate value.

Theorem 2 (Algebraic update soundness). *Assume binding of the Pedersen coefficient, evaluation-vector, membership-vector, and balance commitments; knowledge soundness of the committed-R1CS Bulletproof/IPA for public group-linear relations; injectivity or collision resistance of $\text{Enc}_{\mathbb{F}}$; integer range soundness; and the random-oracle soundness of the Fiat–Shamir challenge β . The adversary may choose the public touched addresses adaptively, subject to the verifier’s distinctness check. Suppose the previous accepted algebraic state binds to a set S and balance B_{i-1} . If the verifier accepts an update from X_{i-1} to X_i , then, except with probability at most $(m-1)/q + \text{negl}(\lambda)$, C_i opens to*

$$B_{i-1} + \sum_{(a, \delta) \in \Delta_i : a \in S} \delta.$$

Proof. Bulletproof knowledge soundness extracts

$$((p_t)_t, u, y, z, w, d_i, r_A, r_U, r_Y, r_D)$$

satisfying the group-linear equations, the batched evaluation equation, and the R1CS constraints. In particular, the extracted opening of A gives a polynomial

$$P(T) = \sum_{t=0}^n p_t T^t.$$

By Pedersen binding and initialization soundness, this is the same scalar-masked set polynomial P_S bound in the previous accepted state. The extracted opening of C_Y gives a vector y . Let

$$e_j = y_j - P_S(x_j).$$

The accepted batched evaluation equation gives

$$\sum_{j=1}^m \beta^{j-1} e_j = 0.$$

The commitments A, C_Y, C_U, C_D and the public update list are fixed before β is derived. If some $e_j \neq 0$, then $E(Z) = \sum_{j=1}^m e_j Z^{j-1}$ is a nonzero polynomial of degree at most $m-1$, and a random β is a root with probability at most $(m-1)/q$. Thus, except for this probability and negligible binding/extraction error, $y_j = P_S(x_j)$ for every j .

The zero-test constraints give

$$u_j \in \{0, 1\}, \quad u_j = 1 \iff y_j = 0.$$

Since $P_S = \alpha F_S$ and $\alpha \neq 0$, this is equivalent to $F_S(x_j) = 0$, hence to $a_j \in S$ except with negligible encoding-collision probability. The linear constraint gives $d_i = \sum_j u_j \delta_j$, and the extracted opening $C_D = d_i V + r_D H_B$ is fixed by Pedersen binding. The verifier checks $C_i = C_{i-1} + C_D$, so the accepted commitment opens to the claimed updated aggregate.

Theorem 3 (SMT update soundness). *Assume SNARK soundness, collision resistance of H , binding of the balance commitment, and integer range soundness. Suppose the previous accepted SMT state binds to a salted reserve tree with aggregate balance B_{i-1} . If the verifier accepts an SMT update from X_{i-1} to X_i , then R_i is the root obtained by applying exactly the public deltas for touched reserve addresses, and C_i opens to the corresponding updated aggregate balance.*

Proof. SNARK soundness extracts a witness satisfying $\mathcal{R}_{\text{upd}}^{\text{smt}}$. For each public touched key, the circuit verifies either a membership path for a salted leaf or a non-membership proof against the current compact root representation. In the membership case it range-checks $b_j + \delta_j$, updates the salted leaf with the same salt, and adds δ_j to the aggregate delta. In the non-membership case it proves absence by a default-subtree or collision-leaf witness and leaves the root and aggregate delta unchanged. Collision resistance prevents two different compact trees, leaves, or paths from producing the same verified root. The circuit recomputes the final root R_i and checks $C_i - C_{i-1} = d_i V + \rho_i H_B$; Pedersen binding fixes the committed value.

Theorem 4 (Insertion soundness). *Assume SNARK soundness, soundness of address-ownership and native chain-state proofs, collision resistance of H , binding of the balance commitment, and integer range soundness. If an SMT insertion proof accepts, then the inserted address is controlled by the prover, was absent from the old tree, its inserted balance equals $\text{bal}_{\text{sr}_i}(a)$, and the new root and balance commitment are the honest insertion result.*

Proof. SNARK soundness extracts a witness satisfying $\mathcal{R}_{\text{ins}}^{\text{smt}}$. Ownership and chain-proof soundness give control of the address and correctness of the inserted balance. The non-membership check proves absence from the old root, except with collision probability for H . The circuit recomputes the inserted salted leaf, the new root, and the balance-commitment difference. Binding fixes the committed aggregate increase.

Theorem 5 (Threshold soundness). *Assume soundness of the comparison proof and binding of the balance and liability commitments. If a threshold proof for the current state accepts, then every valid opening of the checked commitments satisfies the stated integer inequality.*

Proof. The proof system extracts openings used in the comparison relation. Commitment binding makes these openings unique for the public commitments, and range soundness gives integer semantics. Hence an accepted proof implies the claimed inequality for the committed balances.

Theorem 6 (Inductive asset soundness and freshness). *Assume initialization soundness and the corresponding update and insertion soundness theorem for the chosen construction. For any accepted sequence*

$$X_0 \rightarrow X_1 \rightarrow \cdots \rightarrow X_i,$$

where every transition uses the verifier's public synchronizer output between the named finalized roots, C_i opens to the aggregate balance obtained from the initialized hidden reserve set by applying all accepted public updates and insertions. Moreover, the statement is fresh for sr_i : a proof for an older root cannot be accepted for sr_i without an accepted chain of transitions to X_i .

Proof. The base case is initialization soundness. The induction step is Theorem 2 for the algebraic fixed-set construction, or Theorem 3 plus insertion soundness for the SMT construction. The verifier checks that each update references the last accepted state and the synchronizer output for the named roots. Therefore accepted states form a single public chain, and stale states cannot be relabeled as newer roots.

Theorem 7 (Privacy). *Assume zero knowledge of the proof systems, hiding of Pedersen commitments, fresh blindings in A, C_U, C_Y, C_D , and independent SMT leaf salts. Then two executions with the same public leakage and the same threshold outcomes give computationally indistinguishable verifier views.*

Proof. All addresses, balances, paths, membership bits, polynomial coefficients, polynomial evaluations, and commitment openings are private witnesses. Zero knowledge simulates the proofs. Pedersen hiding simulates the coefficient commitment, balance commitments, evaluation-vector commitments, and delta commitments. The batched evaluation challenge β is derived only after hiding commitments are fixed, so it does not reveal the vector y . In the SMT construction, salted leaves prevent the public root from being a deterministic hash of unsalted address-balance pairs. Thus the public view is simulatable from public roots, delta lists, commitments, and accepted predicates.

8 Implementation and Evaluation

This section states the implementation plan and the quantities that must be measured in a prototype. We do not report fabricated benchmark numbers.

Initialization prototype. The prototype should report initialization separately from update. For the algebraic construction it measures native proof verification, address-ownership verification, duplicate-freeness, product-tree construction of P_S , the randomized product check, the Pedersen coefficient-opening proof, and the aggregate commitment check. For the SMT construction it measures native proof verification, salted-leaf construction, compact tree construction, root re-computation, and aggregate commitment checking. This separation is important because initialization is one-time and chain-specific, while updates are repeated frequently.

Algebraic prototype. The prover stores the set polynomial F_S in a representation supporting fast multipoint evaluation. For each m -element update list, the prototype measures multipoint evaluation of P_S , transposed multipoint evaluation for the batching vector b^* , Pedersen commitments to u, y, d , and the merged committed-R1CS Bulletproof/IPA proof. The main baselines are the naive $O(|S|m)$ evaluation strategy and a static SNARK that rechecks all hidden balances.

SMT-SNARK prototype. The circuit uses a SNARK-friendly hash such as Poseidon or Rescue for local tree nodes. It measures independent paths versus compact multiproofs, the number of hash constraints, range constraints, proof generation time, proof size, and verifier time. The relevant parameters are reserve-set size n , update-list length m , number of touched reserve addresses t , and tree depth ℓ .

9 Extensions: Multichain Aggregation

The single-chain state extends naturally to multiple chains. Let tracked chains be $c \in [k]$. The public state contains a vector of finalized roots

$$\mathbf{sr}_i = (\mathbf{sr}_i^{(1)}, \dots, \mathbf{sr}_i^{(k)})$$

Table 1. Asymptotic update costs, excluding one-time initialization.

Construction	Prover work	Proof / verifier work
Algebraic BP/IPA	$\tilde{O}(n + m) + O(n + m)$ group work + $O(m)$ gates	$O(\log(n + m))$ proof, $O(n + m)$ verifier
SMT-SNARK	$O(N_{\text{frontier}})$ hash checks	succinct proof, verifier independent of m
Static SNARK baseline	native proofs for all n addresses	succinct proof, large prover circuit
Public-address audit	public chain scan	no privacy

and a vector of balance commitments

$$\mathbf{C}_i = (C_i^{(1)}, \dots, C_i^{(k)}).$$

The hidden reserve leaf in the SMT construction stores a balance vector $\mathbf{b}(a) = (b^{(1)}(a), \dots, b^{(k)}(a))$. In the algebraic Bulletproof/IPA construction, the membership vector is shared across chains, and the balance-projection constraints are repeated for each chain coordinate or compressed by a Fiat–Shamir random linear combination when appropriate.

Each chain has its own public synchronizer. The merged public delta list contains address-vector-delta pairs $(a, \delta(a))$. The update proof shows

$$B_i^{(c)} = B_{i-1}^{(c)} + \sum_{a \in S} \delta^{(c)}(a) \quad \text{for every } c \in [k].$$

A threshold proof can express a per-chain lower bound, a simple global sum, or a weighted linear form

$$\langle \alpha, \mathbf{B}_i \rangle \geq T.$$

This makes the aggregation semantics explicit and prevents a prover from silently changing what “total assets” means between epochs.

Rollups and bridges require a finality policy. A verifier may use L1-posted roots, native L2 finalized roots, or a hybrid policy. If a rollup reorg invalidates a previously accepted root, the verifier rolls back to the last common accepted state and asks for a new update chain. The cryptographic update proof remains unchanged because it is parameterized only by the public root vector and public synchronizer outputs.

10 Related Work

Proofs of liabilities. Early exchange-audit proposals used Merkle sum trees so that users could verify inclusion of their own liabilities [7]. Maxwell’s proof-of-liabilities idea motivated many later systems, but naive sum-tree constructions can be vulnerable to binding errors if balances are not committed carefully. Follow-up work studied these attacks and proposed fixes and generalizations [8,9]. Recent work on short private proofs of liabilities and accumulator-based systems such as Notus focuses on the liability side [10,11]: Falzon et al. use sparse summation Verkle trees, while Notus gives a dynamic PoL system based

on zero-knowledge RSA accumulators and a MultiSwap protocol. These works motivate our dynamic security style, but their objects are liabilities rather than on-chain reserve balances.

Proofs of assets and solvency. Provisions [6] is the closest classical academic work. It gives privacy-preserving proofs of solvency for Bitcoin exchanges by combining hidden asset proofs with liability proofs. Its setting is essentially static and Bitcoin-oriented. Later SNARK-based and succinct solvency proofs similarly prove ownership of hidden addresses and committed balances in one large statement [13,12]. These schemes do not directly solve the dynamic on-chain setting considered here, where the reserve commitment must be refreshed frequently from public block deltas without rechecking all native state proofs.

Industrial proof of reserves. Many exchange proof-of-reserve systems disclose wallet addresses or combine public asset lists with Merkle or ZK liability proofs. This gives transparency but not reserve-set privacy. Our work targets the missing case: an on-chain asset proof that keeps reserve addresses hidden while still providing continuous updates tied to finalized chain roots.

Authenticated state and zero knowledge. Merkle trees, sparse Merkle trees, and Ethereum-style Merkle-Patricia tries are standard tools for authenticated state [4,14,17]. They are not equally convenient inside zero-knowledge circuits. Our SMT construction deliberately uses a circuit-friendly local tree rather than the native chain tree after initialization. Bulletproofs and inner-product arguments [2] provide the algebraic tools for the fixed-set construction: Pedersen vector commitments bind polynomial coefficients and committed evaluation vectors, while a random-linear-combination Bulletproof/IPA proof gives batched polynomial evaluation and committed arithmetic relations without pairings. KZG commitments [1] remain a useful alternative when evaluation values are public and a structured setup is acceptable, but they are less direct when the output evaluations are themselves hidden Pedersen commitments.

11 Conclusion

We introduced dynamic private proof of assets for hidden on-chain reserve sets. The central idea is to separate the expensive initialization proof from cheap incremental updates driven by public block-delta lists. We gave two concrete constructions: a specialized Bulletproof/IPA update protocol for fixed reserve sets and a compact SMT-SNARK construction for mutable reserve sets. Both bind aggregate asset commitments to finalized chain state roots while hiding reserve addresses and aggregate deltas. Future work includes a full implementation, optimized circuits for compact multiproofs, and a detailed empirical comparison with static SNARK-based audits.

References

1. A. Kate, G. M. Zaverucha, and I. Goldberg. Constant-size commitments to polynomials and their applications. In *ASIACRYPT*, 2010.
2. B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE Symposium on Security and Privacy*, 2018.
3. J. Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.
4. R. C. Merkle. A digital signature based on a conventional encryption function. In *CRYPTO*, 1987.
5. L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *USENIX Security Symposium*, 2021.
6. G. G. Dagher, B. Bünz, J. Bonneau, J. Clark, and D. Boneh. Provisions: Privacy-preserving proofs of solvency for Bitcoin exchanges. In *ACM CCS*, 2015.
7. G. Maxwell. Proving your Bitcoin reserves. Bitcoin Forum post, 2014.
8. K. Hu, Z. Zhang, and D. Lin. Breaking the binding: Attacks on the Merkle approach to proving liabilities and its applications. Cryptology ePrint Archive, Report 2018/1139, 2018.
9. Y. Ji and K. Chalkias. Generalized proof of liabilities. In *Financial Cryptography and Data Security*, 2021.
10. F. Falzon, K. Elkhyaoui, Y. Manevich, and A. De Caro. Short privacy-preserving proofs of liabilities. In *ACM CCS*, 2023.
11. J. Xin, A. Haghighi, X. Tian, and D. Papadopoulos. Notus: Dynamic proofs of liabilities from zero-knowledge RSA accumulators. In *USENIX Security Symposium*, 2024.
12. Y. Deng and J. Clark. Xiezhi: Toward succinct proofs of solvency. Cryptology ePrint Archive, Paper 2024/2001, 2024.
13. B. S. Reddy. A ZK-SNARK based proof of assets protocol for Bitcoin exchanges. arXiv:2208.01263, 2022.
14. G. Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper, 2014–2024.
15. J. Kuszmaul. Verkle trees. Cryptology ePrint Archive, Report 2018/923, 2018.
16. E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 2014.
17. Ethereum Foundation. Merkle Patricia Trie. Ethereum developer documentation.
18. M. Kirejczyk, M. Kalka, and L. Logvinov. Historical and multichain storage proofs. arXiv:2411.00193, 2024.

A Full Bulletproof/IPA Specification

This appendix expands the proof denoted $\pi_{\text{upd}}^{\text{bp}}$. The witness vector is

$$a = ((p_t)_{t=0}^n, u_1, \dots, u_m, y_1, \dots, y_m, z_1, \dots, z_m, w_1, \dots, w_m, d, r_A, r_U, r_Y, r_D).$$

The public group-linear equations are

$$A = \sum_{t=0}^n p_t G_t + r_A H_A,$$

$$C_U = \sum_j u_j U_j + r_U H_U, \quad C_Y = \sum_j y_j Y_j + r_Y H_Y, \quad C_D = dV + r_D H_B.$$

The public linear equations are

$$\sum_{t=0}^n p_t b_t^* - \sum_{j=1}^m \gamma_j y_j = 0, \quad d - \sum_j \delta_j u_j = 0.$$

For each j , the four multiplication gates are

$$u_j(1 - u_j) = 0, \quad u_j y_j = 0, \quad y_j z_j = w_j, \quad (1 - u_j)(w_j - 1) = 0.$$

A standard committed-R1CS Bulletproof aggregates these constraints and reduces the final check to an inner-product argument. The coefficient-opening equation for A increases the committed-vector length by $n + 1$, but it adds no multiplication gates.

For completeness, recall the inner-product argument. Let N be a power of two, let $G = (G_1^*, \dots, G_N^*)$ and $H = (H_1^*, \dots, H_N^*)$, and let U^* be an additional generator. The public instance is

$$P = \langle a, G \rangle + \langle b, H \rangle + \langle a, b \rangle U^*.$$

If $N > 1$, split vectors into left and right halves and send

$$L = \langle a_L, G_R \rangle + \langle b_R, H_L \rangle + \langle a_L, b_R \rangle U^*, \quad R = \langle a_R, G_L \rangle + \langle b_L, H_R \rangle + \langle a_R, b_L \rangle U^*.$$

After challenge x , both parties fold

$$a' = x a_L + x^{-1} a_R, \quad b' = x^{-1} b_L + x b_R,$$

$$G' = x^{-1} G_L + x G_R, \quad H' = x H_L + x^{-1} H_R, \quad P' = P + x^2 L + x^{-2} R.$$

The protocol recurses until one coordinate remains. The proof contains $(L_1, R_1, \dots, L_{\log N}, R_{\log N}, a_{\text{final}}, b_{\text{final}})$.

B Concrete R1CS Rows

For each update index j , the zero-test relation can be written as R1CS rows $\ell(a)r(a) = o(a)$:

gate	$\ell(a)$	$r(a)$	$o(a)$
bit	u_j	$1 - u_j$	0
zero if member	u_j	y_j	0
inverse test	y_j	z_j	w_j
nonzero if nonmember	$1 - u_j$	$w_j - 1$	0

If $u_j = 1$, the second row forces $y_j = 0$. If $u_j = 0$, the fourth row forces $w_j = 1$, and the third row then gives $y_j z_j = 1$, hence $y_j \neq 0$. Together with the first row, this proves $u_j = 1$ iff $y_j = 0$.

C Omitted Proof Details

Batched multi-evaluation soundness. The prover first fixes the commitments A, C_U, C_Y, C_D and the public state transition. The Fiat–Shamir challenge β is then derived from this transcript. Binding of A and C_Y fixes a coefficient vector p and a claimed evaluation vector y . If y is not the true evaluation vector of $P(T) = \sum_t p_t T^t$ at $(x_j)_j$, then the error polynomial

$$E(Z) = \sum_{j=1}^m (y_j - P(x_j)) Z^{j-1}$$

is nonzero and has degree at most $m-1$. The batched equation is exactly $E(\beta) = 0$. Therefore a false vector passes with probability at most $(m-1)/q$, plus the negligible probability of breaking the commitment binding or the Bulletproof extractor. This replaces the KZG batch-opening binding argument in the pairing-based version.

Simulation. The algebraic simulator chooses random blinded commitments A, C_U, C_Y, C_D consistent with the public commitment update and simulates the zero-knowledge Bulletproof transcript. Since A and C_Y are Pedersen commitments with fresh blindings, the simulator does not need to know the hidden set polynomial or the evaluation vector. The SMT simulator outputs simulated SNARK proofs and random hiding commitments. Indistinguishability follows from the zero-knowledge property of the underlying proof systems and the hiding of the commitments.

D Multipoint and Transposed Evaluation Details

The prover should not compute $P_S(x_j)$ independently for all j , which would cost $O(nm)$. Instead it builds the subproduct tree for the query polynomial $Z_i(T) = \prod_j (T - x_j)$, reduces P_S down the tree, and reads the remainders at the leaves. With FFT-based polynomial arithmetic this costs $\tilde{O}(n+m)$; a conservative bound for straightforward implementations is $O((n+m) \log^2(n+m))$.

The batching vector

$$b_t^* = \sum_{j=1}^m \beta^{j-1} x_j^t$$

is the transpose of multipoint evaluation. It can be computed with the transposed subproduct-tree algorithm in the same asymptotic polynomial-arithmetic complexity as multipoint evaluation. If the query points lie in a roots-of-unity domain, both y and b^* can be computed with FFT/NTT routines, giving the optimized $O(n \log n + m \log m)$ -style cost used in structured-domain deployments. If m is very small, the implementation may instead use the simple recurrence $s_j \leftarrow \beta^{j-1}$ and Horner-style accumulation, which costs $O(nm)$ but has lower constants.

E SMT Non-Membership Cases

A sparse Merkle non-membership proof has two circuit cases.

Default-subtree case. The path for key κ reaches a subtree whose root equals the precomputed default hash E_d . Since every leaf below this node is empty, κ is absent. The circuit checks the path from this default node to the root.

Collision-leaf case. The path reaches a non-empty leaf with key $\kappa' \neq \kappa$. The circuit checks that κ' diverges from κ at the claimed bit, recomputes the leaf hash for κ' , and verifies the path to the root. Collision resistance of H prevents the prover from using the same root for two inconsistent leaf assignments.